

Exercise 1: Implementing the Singleton Pattern

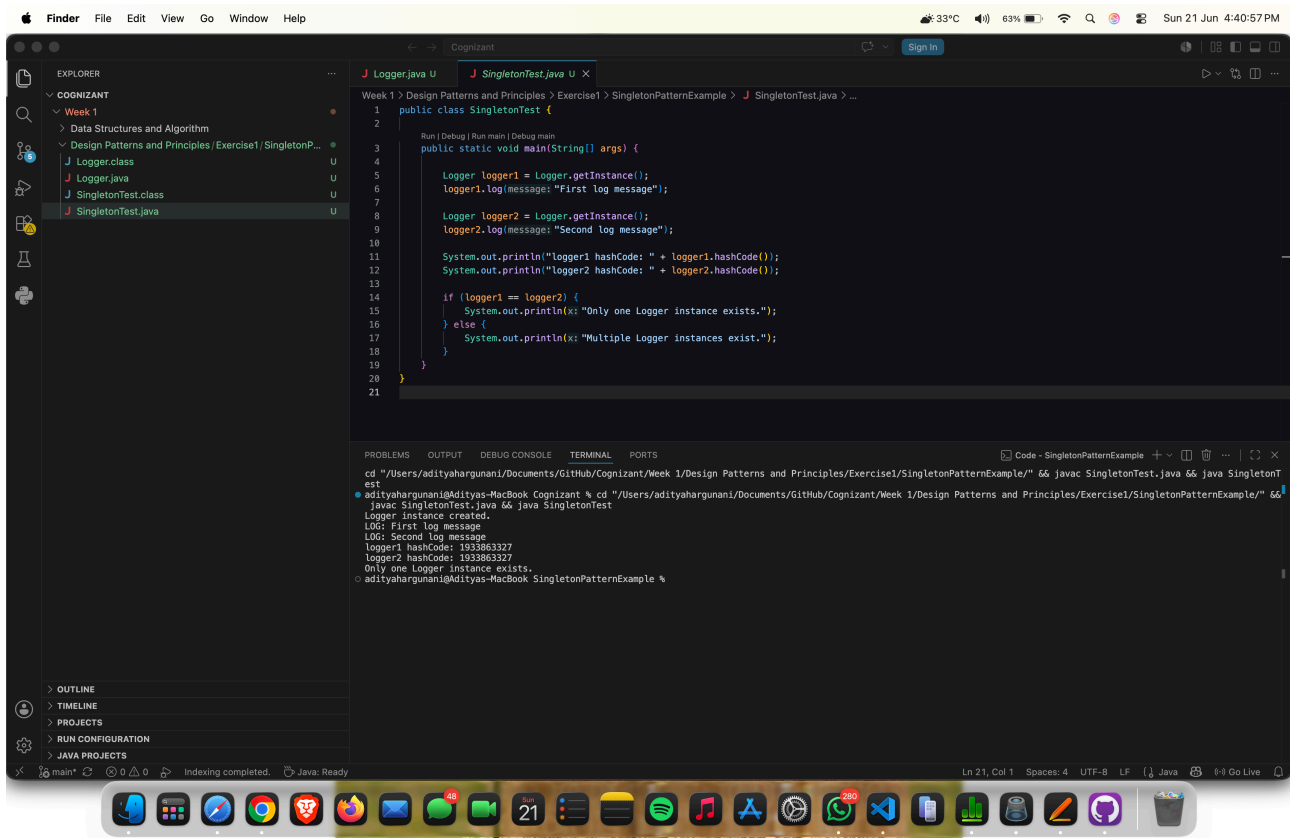
Logger.java:

```
public class Logger {  
    private static Logger instance;  
  
    private Logger() {  
        System.out.println("Logger instance created.");  
    }  
  
    public static Logger getInstance() {  
        if (instance == null) {  
            instance = new Logger();  
        }  
        return instance;  
    }  
  
    public void log(String message) {  
        System.out.println("LOG: " + message);  
    }  
}
```

SingletonTest.java:

```
public class SingletonTest {  
    public static void main(String[] args) {  
        Logger logger1 = Logger.getInstance();  
        logger1.log("First log message");  
  
        Logger logger2 = Logger.getInstance();  
        logger2.log("Second log message");  
  
        System.out.println("logger1 hashCode: " + logger1.hashCode());  
        System.out.println("logger2 hashCode: " + logger2.hashCode());  
  
        if (logger1 == logger2) {  
            System.out.println("Only one Logger instance exists.");  
        } else {  
            System.out.println("Multiple Logger instances exist.");  
        }  
    }  
}
```

Output:



The screenshot displays an IDE window with a project named 'Cognizant'. The Explorer panel on the left shows the project structure, including 'Week 1' and 'Design Patterns and Principles/Exercise1/SingletonPatternExample'. The main editor shows the 'SingletonTest.java' file, which implements a Singleton pattern. The code defines a 'Logger' class with a static 'getInstance()' method and a 'SingletonTest' class with a 'main()' method that creates two logger instances and prints their hash codes. The output console at the bottom shows the execution results, confirming that only one logger instance exists.

```
Week 1 > Design Patterns and Principles > Exercise1 > SingletonPatternExample > SingletonTest.java > ...
1 public class SingletonTest {
2
3     Run | Debug | Run main | Debug main
4     public static void main(String[] args) {
5
6         Logger logger1 = Logger.getInstance();
7         logger1.log(message: "First log message");
8
9         Logger logger2 = Logger.getInstance();
10        logger2.log(message: "Second log message");
11
12        System.out.println("logger1 hashCode: " + logger1.hashCode());
13        System.out.println("logger2 hashCode: " + logger2.hashCode());
14
15        if (logger1 == logger2) {
16            System.out.println(k: "Only one Logger instance exists.");
17        } else {
18            System.out.println(k: "Multiple Logger instances exist.");
19        }
20    }
21 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
cd "/Users/adityahargunani/Documents/GitHub/Cognizant/Week 1/Design Patterns and Principles/Exercise1/SingletonPatternExample/" && javac SingletonTest.java && java SingletonTest
adityahargunani@Adityas-MacBook: Cognizant % cd "/Users/adityahargunani/Documents/GitHub/Cognizant/Week 1/Design Patterns and Principles/Exercise1/SingletonPatternExample/" &&
javac SingletonTest.java && java SingletonTest
Logger instance created.
LOG: First log message
LOG: Second log message
logger1 hashCode: 1933863327
logger2 hashCode: 1933863327
Only one Logger instance exists.
adityahargunani@Adityas-MacBook: SingletonPatternExample %
```